

Part-Of-Speech Tagging, Sequence Labeling

Lecturer: Doctor Bui Thanh Hung

Data Science Laboratory

Faculty of Information Technology

Industrial University of Ho Chi Minh city

Email: hung.buithanhcs@gmail.com)

Website: <https://sites.google.com/site/hungthanhbui1980/>

Exercise 1:

Download the dataset Alice's Adventures in Wonderland (written by Lewis Carroll) distributed on Project Gutenberg, as a text file `alice.txt`. Apply a part-of-speech (POS) tagger to the text file, and store the result in another file. Implement programs that read the POS tagging result and perform the following requirements:

1.1 Implement a program that reads the result of part-of-speech tagging. Here, represent a sentence as a list of mapping objects, each of which associates a surface form, lemma (base form), part-of-speech tag with the keys `text`, `lemma`, `pos`. Use this representation in the rest of the problems.

1.2 Extract surface forms of all verbs appearing in the text.

1.3. Extract lemmas of all verbs appearing in the text.

1.4 Extract noun phrases in the form of "A of B", where A and B are nouns.

1.5 Extract the longest noun phrase consisting of consecutive nouns.

1.6 Obtain the list of words and frequencies of their occurrences sorted by descending order of frequency.

1.7 Visualize the top-ten frequent words and their frequencies with a chart (e.g., bar chart).

1.8 Extract the list of words that co-occur with the word "Alice". Visualize with a chart (e.g., bar chart) the top-ten words co-occurring with the word "Alice" and their frequencies.

1.9 Draw a histogram of word frequency (x-axis is a scalar range representing a frequency ranging from 1 to the largest frequency of a given word in the entire corpus, and the y-axis is the count of unique words that fall into the count of the x value).

1.10. Plot a log-log graph with the x-axis being rank order and the y-axis being frequency (Zipf's law).

Exercise 2:

Build the Viterbi algorithm for below dataset

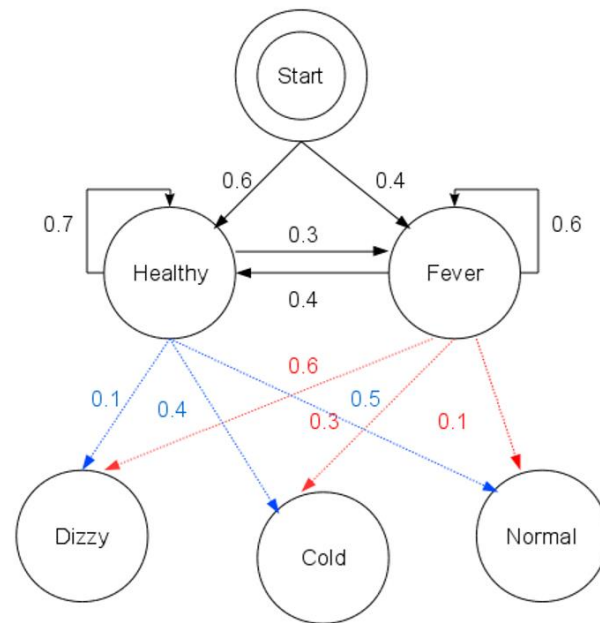
Viterbi

```
obs = ('normal', 'cold', 'dizzy')
states = ('Healthy', 'Fever')
start_p = {'Healthy': 0.6, 'Fever': 0.4}
trans_p = {
    'Healthy': {'Healthy': 0.7, 'Fever': 0.3},
    'Fever': {'Healthy': 0.4, 'Fever': 0.6}
}
emit_p = {
    'Healthy': {'normal': 0.5, 'cold': 0.4, 'dizzy': 0.1},
    'Fever': {'normal': 0.1, 'cold': 0.3, 'dizzy': 0.6}
}
```

day 1: cold

day 2: dizzy

day 3: normal



Exercise 3:

Build the above Viterbi algorithm by yourself

Exercise 4:

Read a slide and implement the rule-based method for sequence labeling tasks